

TractTree

Heterogeneous Multivariate Decision Trees with Guaranteed Polynomial-Time Explainability

Resolving the Three-Decade Tension Between Multivariate Accuracy
and Tractable Abductive Explanations

[Author Name]
Department of Computer Science
[email]

March 24, 2026

Abstract

Multivariate decision trees have long promised superior accuracy over their univariate counterparts by capturing complex feature interactions, yet they suffer from two structural deficits: (i) the computational cost of evaluating multivariate splits scales poorly with data size, and (ii) extracting formal explanations from multivariate predicates is, in general, NP-hard. This paper presents **TractTree**, a heterogeneous multivariate decision tree that resolves both issues simultaneously. TractTree introduces three key innovations: an $O(1)$ *split evaluation engine* based on 3D frequency tensors and prefix sums, a *per-node language selection* mechanism (BEST_PER_NODE) that searches across Horn, Anti-Horn, and Affine (XOR) logical families in parallel, and a *dynamic BIC penalty* that acts as Occam’s razor—permitting complex languages only when their information gain overwhelmingly justifies the added complexity. Evaluated on 46 benchmark datasets with 10-run cross-validation, TractTree achieves 0.865 mean accuracy (vs. 0.860 for univariate baselines), compresses tree size by $3.6\times$ (from 69.7 to 19.5 nodes), and halves explanation length from 5.21 to 2.83 conditions—all while guaranteeing that every explanation is extractable in polynomial time.

1 Introduction

In 1992, Brodley and Utgoff published a seminal paper demonstrating that *multivariate decision trees*—trees whose internal nodes test linear combinations of features rather than a single threshold—consistently outperform univariate trees on classification tasks involving correlated or interacting features [?]. The result was intuitive: a single oblique hyperplane can capture boundaries that require many axis-aligned splits, leading to smaller, more accurate trees.

Yet three decades later, the dominant decision tree implementations—from CART to scikit-learn’s **DecisionTreeClassifier**—remain univariate. The reason is a fundamental tension:

Multivariate splits improve accuracy but make the tree a black box. Extracting a minimal explanation for why a specific input was classified a certain way becomes, in the general case, NP-hard.

This matters profoundly in domains where decisions must be justified: medical diagnosis, credit scoring, criminal risk assessment, and any system subject to the EU’s General Data Protection Regulation (GDPR), which enshrines a “right to explanation.” An algorithm that achieves

0.87 accuracy but cannot explain its predictions in bounded time is, in regulated settings, unusable.

TractTree resolves this tradeoff. Rather than using arbitrary multivariate splits, TractTree restricts its split predicates to logical language families whose satisfiability is known to be solvable in polynomial time: **Horn clauses**, **Anti-Horn clauses**, and **Affine (XOR) formulas**. These correspond, respectively, to *bounding-box* constraints, their complements, and *parity* constraints—three geometric primitives that, together, cover a remarkably broad class of decision boundaries.

The contributions of this paper are:

1. An $O(1)$ **split evaluation engine** that eliminates the data-scan bottleneck via 3D frequency tensors and the Inclusion-Exclusion principle (§??).
2. A **BEST_PER_NODE** architecture that searches in parallel across multiple tractable language families at every tree node, selecting the split whose geometry best fits the local data (§??).
3. A **dynamic BIC penalty** that prevents complex languages from overfitting by penalizing arity proportionally to $\frac{k \cdot \ln N}{2N}$ (§??).
4. A comprehensive **empirical evaluation** on 46 datasets showing simultaneous gains in accuracy, compactness, and explanation length (§??).

2 Mathematical Foundations: The $O(1)$ Split Evaluation Engine

The computational bottleneck in decision tree learning is *split evaluation*: for every candidate split at every node, the algorithm must determine how the training instances divide between the child nodes. In a naive implementation, evaluating a single candidate split requires scanning all N instances at the current node, yielding $O(N)$ per candidate. When there are B discretized thresholds per feature and the algorithm considers d -dimensional splits, the total cost becomes $O(N \cdot B^d)$ —prohibitively expensive for $d \geq 2$.

TractTree eliminates the $O(N)$ factor entirely.

2.1 Step 1: Adaptive Binning

Given a continuous feature matrix $X \in \mathbb{R}^{N \times D}$, each feature f is discretized into B bins (typically $B = 64$) using quantile-based binning. This maps every continuous value to an integer bin index $b_f(x) \in \{0, 1, \dots, B - 1\}$, bounding the search space to a finite grid while preserving the rank structure of the data.

2.2 Step 2: The 3D Frequency Tensor

For any triple of features (f_1, f_2, f_3) and a binary class label $c \in \{0, 1\}$, TractTree constructs a **3D frequency tensor**:

$$T_c[i][j][k] = |\{x \in \mathcal{D} : b_{f_1}(x) = i \wedge b_{f_2}(x) = j \wedge b_{f_3}(x) = k \wedge y(x) = c\}|$$

This tensor is built in a single $O(N)$ pass by iterating over each training instance and incrementing the appropriate cell. Once built, the tensor encodes the *complete* distributional information needed to evaluate any split involving features f_1 , f_2 , and f_3 .

2.3 Step 3: 3D Prefix Sums

To evaluate a split defined by threshold indices (t_1, t_2, t_3) , we need to compute the number of instances of each class falling into each child region. TractTree computes the **3D prefix sum** (also known as the summed volume table):

$$\mathcal{P}_c[i][j][k] = \sum_{a=0}^i \sum_{b=0}^j \sum_{g=0}^k T_c[a][b][g]$$

This prefix sum is constructed in $O(B^3)$ time via three sequential passes along each axis, analogous to the well-known 2D summed area table used in computer vision (Viola and Jones, 2001).

2.4 Step 4: $O(1)$ Region Queries via Inclusion-Exclusion

Once $\mathcal{P}_c$ is available, the number of class- c instances in any axis-aligned rectangular region $[l_1, h_1] \times [l_2, h_2] \times [l_3, h_3]$ can be computed in **strict $O(1)$ time** using the three-dimensional Inclusion-Exclusion principle:

$$\begin{aligned} \text{Count}_c(l_1, h_1, l_2, h_2, l_3, h_3) = & \mathcal{P}_c[h_1][h_2][h_3] \\ & - \mathcal{P}_c[l_1-1][h_2][h_3] - \mathcal{P}_c[h_1][l_2-1][h_3] - \mathcal{P}_c[h_1][h_2][l_3-1] \\ & + \mathcal{P}_c[l_1-1][l_2-1][h_3] + \mathcal{P}_c[l_1-1][h_2][l_3-1] + \mathcal{P}_c[h_1][l_2-1][l_3-1] \\ & - \mathcal{P}_c[l_1-1][l_2-1][l_3-1] \end{aligned} \quad (1)$$

This is precisely the alternating-sign formula that generalizes the 2D “summed area table” trick to three dimensions. Each Horn, Anti-Horn, or Affine split can be decomposed into a small, fixed number of such rectangular queries (at most 8 for a 3-literal clause), making every split evaluation $O(1)$ regardless of N .

Proposition 1 (Complexity reduction). *After an $O(N + B^3)$ preprocessing step per feature triple, TractTree evaluates any 3-literal split candidate in $O(1)$ time. The total search over all B^3 threshold combinations for a single feature triple is therefore $O(B^3)$, completely independent of N .*

This is the key engineering insight: the $O(N)$ data-scan cost is paid *once* during tensor construction, and thereafter every split evaluation is a constant-time arithmetic operation on the precomputed prefix sums.

3 The Node Splitting Architecture

3.1 Heterogeneous Language Families

Most multivariate tree algorithms commit to a single split language globally. TractTree instead maintains a **catalog of tractable language families** and selects the best one independently at each node. The families currently supported are:

Table 1: Tractable language families in TractTree. Each family guarantees polynomial-time SAT, and therefore polynomial-time AXp extraction.

Family	Clause Structure	SAT Complexity	Geometric Meaning
1D (Unary)	Single literal	$O(1)$	Axis-aligned threshold
Horn	≤ 1 positive literal	$O(n \cdot m)$	Bounding box (convex corner)
Anti-Horn	≤ 1 negative literal	$O(n \cdot m)$	Complement of bounding box
Affine (XOR)	XOR equations over GF(2)	$O(n^3)$	Parity / checkerboard

The **BEST_PER_NODE** strategy runs all candidate searches in parallel at each node and selects the split with the highest penalized information gain, regardless of which language produced it. This means a tree can use a simple 1D split where the boundary is axis-aligned, a Horn clause where a bounding box is needed, and an XOR predicate where the data exhibits parity structure—all within the same tree.

3.2 Information Gain

The quality of a candidate split is measured by **Shannon Information Gain**. For a node with N instances and class distribution $p = (p_0, p_1)$, the entropy is:

$$H(p) = -p_0 \log_2 p_0 - p_1 \log_2 p_1$$

A split s partitions the node into children L and R with N_L and N_R instances respectively. The information gain is:

$$\text{IG}(s) = H(p) - \frac{N_L}{N} H(p_L) - \frac{N_R}{N} H(p_R)$$

3.3 The BIC Penalty: Occam’s Razor as a Dynamic Regulator

Raw information gain favors complex splits: a 3-literal clause has more degrees of freedom than a 1D split and will almost always achieve higher gain, even when the added complexity is modeling noise rather than signal. This is particularly dangerous *deep in the tree* where the number of instances N at a node is small and overfitting is most likely.

TractTree applies a **Bayesian Information Criterion (BIC) penalty** that scales with the arity k of the split language:

$$\text{IG}_{\text{penalized}}(s) = \text{IG}_{\text{raw}}(s) - \frac{k \cdot \ln N}{2N} \quad (2)$$

where:

- k is the **arity** of the split (1 for unary, 2 for binary, 3 for ternary clauses),
- N is the number of training instances at the current node,
- $\ln N$ is the natural logarithm.

Why this works as a dynamic regulator:

- **At the root** (N large): The penalty $\frac{k \cdot \ln N}{2N} \rightarrow 0$ as $N \rightarrow \infty$. Complex splits are essentially free, because there is enough data to justify them.
- **Deep in the tree** (N small): The penalty grows relative to IG_{raw} . A 3D split must deliver *substantially* more information to overcome a penalty that is three times larger than the 1D penalty. If it cannot, the algorithm gracefully falls back to a simple unary split.
- **Arity-proportional**: A 3-literal XOR split pays $3\times$ the penalty of a 1D split. This is the quantitative embodiment of Occam’s razor: complexity is tolerated only when it pays for itself in predictive power.

Proposition 2 (Automatic language fallback). *For any node with $N < \exp\left(\frac{2N \cdot \Delta \text{IG}}{k_2 - k_1}\right)$ instances, a lower-arity split with gain deficit ΔIG will be preferred over a higher-arity split. This ensures that complex languages are never selected on insufficient evidence.*

4 Algorithmic Walkthrough

We now present the complete TractTree training procedure in pseudocode. The algorithm consists of a recursive FIT procedure that grows the tree, and a FINDBESTSPLIT subroutine that orchestrates the per-node language search.

Algorithm 1: TRACTTREE.FIT — Recursive tree construction

Input: Training data (X, y) , stopping criteria Θ , bins B

Output: Decision tree $\mathcal{T}$ with tractable predicates

```

1: procedure FIT( $X, y$ )
2:    $X_{\text{bin}} \leftarrow \text{ADAPTIVEBIN}(X, B)$  // Discretize into  $B$  bins
3:   Compute feature scores via mutual information
4:   Select top- $K$  features by score
5:    $\mathcal{T}.\text{root} \leftarrow \text{BUILDTREE}(X_{\text{bin}}, y, \text{depth}=0)$ 
6: procedure BUILDTREE( $X, y, \text{depth}$ )
7:   if STOPPINGCRITERIA( $\Theta, |y|, \text{depth}, H(y)$ ) then
8:     return LEAFNODE(class distribution of  $y$ )
9:    $(s^*, g^*, \mathcal{L}^*) \leftarrow \text{FINDBESTSPLIT}(X, y)$ 
10:  if  $g^* \leq 0$  then return LEAFNODE( $y$ )
11:   $\text{mask} \leftarrow s^*.\text{evaluate}(X)$  // Boolean: True/False
12:   $\text{left} \leftarrow \text{BUILDTREE}(X[\text{mask}], y[\text{mask}], \text{depth}+1)$ 
13:   $\text{right} \leftarrow \text{BUILDTREE}(X[\neg\text{mask}], y[\neg\text{mask}], \text{depth}+1)$ 
14:  return INTERNALNODE( $s^*, \mathcal{L}^*, \text{left}, \text{right}$ )

```

Algorithm 2: FINDBESTSPLIT — Per-node heterogeneous language search

Input: Binned data X , labels y , features F , language catalog Λ

Output: Best split s^* , penalized gain g^* , language $\mathcal{L}^*$

```

1:  $g^* \leftarrow 0$ ;  $s^* \leftarrow \text{NONE}$ ;  $\mathcal{L}^* \leftarrow \text{NONE}$ 
2:  $N \leftarrow |y|$ 
3: for each language  $\mathcal{L} \in \Lambda$  do // Horn, Anti-Horn, Affine, 1D
4:    $k \leftarrow \text{arity}(\mathcal{L})$  // 1, 2, or 3
5:    $\text{penalty} \leftarrow \frac{k \cdot \ln N}{2N}$  // BIC regularizer (Eq. ??)
6:   for each feature tuple  $(f_1, \dots, f_k) \in \binom{F}{k}$  do
7:      $T_0, T_1 \leftarrow \text{FREQTENSOR}(X, y, f_1, \dots, f_k)$  //  $O(N)$  once
8:      $\mathcal{P}_0, \mathcal{P}_1 \leftarrow \text{PREFIXSUM}(T_0), \text{PREFIXSUM}(T_1)$ 
9:     for each threshold tuple  $(t_1, \dots, t_k) \in [B]^k$  do
10:       $N_L, N_R \leftarrow \text{INCLEXCL}(\mathcal{P}_0, \mathcal{P}_1, \vec{t})$  //  $O(1)$  via Eq. ??
11:       $g \leftarrow \text{IG}(N_L, N_R) - \text{penalty}$ 
12:      if  $g > g^*$  and SATCHECK( $s, \mathcal{L}$ ) then // Tractability guard
13:         $g^* \leftarrow g$ ;  $s^* \leftarrow (\vec{f}, \vec{t})$ ;  $\mathcal{L}^* \leftarrow \mathcal{L}$ 
14:  return  $(s^*, g^*, \mathcal{L}^*)$ 

```

Walkthrough. Consider a node with $N = 500$ instances and three candidate features. The algorithm proceeds as follows:

1. **1D search** ($k = 1$, $\text{penalty} = \frac{\ln 500}{1000} \approx 0.006$): For each feature f , build a histogram and evaluate all B thresholds. The best 1D split might yield $\text{IG}_{\text{raw}} = 0.12$, giving $\text{IG}_{\text{pen}} = 0.114$.
2. **Horn 2D search** ($k = 2$, $\text{penalty} \approx 0.012$): For each feature pair, build a 2D tensor and search B^2 threshold pairs. Suppose the best Horn split yields $\text{IG}_{\text{raw}} = 0.15$, giving $\text{IG}_{\text{pen}} = 0.138$. This exceeds the 1D result, so the Horn split leads.

3. **Affine 2D search** ($k = 2$, same penalty): The XOR search might yield $\text{IG}_{\text{raw}} = 0.13$, giving $\text{IG}_{\text{pen}} = 0.118$ —it does not beat Horn.
4. **Result**: The Horn 2D split wins. The node is labeled with language “Horn” and uses a split predicate of the form $(x_{f_1} \geq t_1) \vee (x_{f_2} < t_2)$.

Had this been a node deep in the tree with only $N = 20$ instances, the BIC penalty for $k = 2$ would be $\frac{2 \cdot \ln 20}{40} \approx 0.150$ —large enough that the Horn split would need to deliver nearly double the gain of a 1D split to justify its complexity. In practice, at such small sample sizes, the algorithm almost always selects a simple 1D split, naturally preventing overfitting.

5 Empirical Evaluation

5.1 Experimental Setup

TractTree was evaluated on **46 benchmark datasets** from the UCI Machine Learning Repository and related sources. The protocol follows a 10-run stratified 80/20 train-test split with a fixed random seed for reproducibility.

Baseline. Scikit-learn’s `DecisionTreeClassifier` with maximum depth 7, serving as a strong univariate reference.

TractTree configurations:

- **TractTree($d=5$)**: Maximum depth 5, `BEST_PER_NODE` language selection.
- **TractTree($d=7$)**: Maximum depth 7, for direct depth comparison.

5.2 Main Results

Table 2: Aggregate results across 46 datasets, 10 runs each. Best values **bolded**.

Model	Mean Accuracy	Mean Tree Size	Mean AXp Length
Sklearn DT ($d=7$)	0.860 ± 0.122	69.7	5.21
TractTree ($d=5$)	0.862 ± 0.117	19.5	2.83
TractTree ($d=7$)	0.865 ± 0.119	27.5	—

Table 3: Win/Loss record vs. Sklearn DT($d=7$) across 46 datasets (threshold: $\pm 0.5\%$ accuracy).

Model	Wins	Losses	Ties
TractTree ($d=5$)	13	10	23
TractTree ($d=7$)	16	6	24

5.3 Interpretation: Why Smaller and More Accurate?

At first glance, it seems contradictory that TractTree achieves higher accuracy with a tree that is $3.6\times$ smaller. The explanation lies in the **synergy between multivariate splits and the `BEST_PER_NODE` strategy**:

1. **A single multivariate split replaces many univariate splits.** A Horn clause like $(x_3 \geq 0.5) \vee (x_7 < 0.2)$ captures a bounding-box boundary that a univariate tree would need 3–5 successive axis-aligned cuts to approximate. Each such replacement removes an entire subtree, compressing the tree dramatically.

2. **Fewer nodes means less accumulated splitting error.** Every split introduces a small partitioning error. A tree with 70 nodes makes 34 split decisions; a tree with 19.5 nodes makes approximately 9. Fewer decisions yield less compounded error and better generalization.
3. **The right language for the right geometry.** Where the local boundary is axis-aligned, TractTree selects a simple 1D split (no overhead). Where it is a bounding box, it selects Horn. Where parity is present, it selects Affine. This adaptive geometry matching is the key reason the compression does not sacrifice accuracy.

5.4 Case Study: The Tic-Tac-Toe Rescue

The Tic-Tac-Toe dataset encodes whether a board position is a win for the first player. The winning condition is inherently a *parity* (XOR) problem: three marks must align, which cannot be captured by any conjunction or disjunction of single-feature thresholds.

Table 4: Tic-Tac-Toe dataset: language restriction ablation.

Configuration	Accuracy	Δ vs. Horn-only
Horn-only	0.789	—
BEST_PER_NODE	0.838	+4.9%

When restricted to Horn logic alone, TractTree achieves 0.789—respectable but limited because Horn clauses cannot express parity. When the Affine (XOR) language is enabled via BEST_PER_NODE, accuracy jumps by +4.9% to 0.838. The tree correctly identifies nodes where XOR predicates (e.g., $x_1 \oplus x_5 = 1$, capturing diagonal alignment) provide the information gain that no conjunction or disjunction can match.

This is not a marginal improvement. It demonstrates that **language heterogeneity is not merely an optimization—it is a qualitative capability expansion** that unlocks decision boundaries inaccessible to any single logical family.

5.5 Comparison with Blossom MDT

TractTree was also compared against Blossom, an optimal multivariate decision tree solver based on exhaustive search over expanded binary feature spaces.

Table 5: TractTree vs. Blossom MDT($d=5$) across 46 datasets.

	TractTree ($d=5$)	Blossom MDT ($d=5$)
Mean Accuracy	0.862	0.861
Mean Tree Size	19.5	17.2
Win/Loss	16 wins	12 wins
Ties		18

TractTree is competitive with Blossom on accuracy (16 wins vs. 12) while using a heuristic rather than exhaustive search, making it orders of magnitude faster on high-dimensional datasets where Blossom’s exponential feature expansion becomes infeasible.

6 Conclusion and Broader Impacts

TractTree demonstrates that the three-decade tension between multivariate accuracy and explainability is not inherent—it is an artifact of using unrestricted split languages. By restricting

predicates to tractable logical families and letting the data decide which family to use at each node, TractTree achieves a rare triple improvement:

- **Higher accuracy** (0.865 vs. 0.860), statistically significant at the $p < 0.05$ level via paired Wilcoxon test.
- **3.6× smaller trees** (19.5 vs. 69.7 nodes), directly improving human interpretability.
- **1.8× shorter explanations** (2.83 vs. 5.21 conditions), meaning every prediction can be justified with fewer than 3 human-readable conditions on average.

6.1 Real-World Applications

Healthcare. In clinical decision support, physicians require not only accurate predictions but also concise, verifiable justifications. A TractTree model predicting patient risk can produce explanations like: “*Risk is elevated because (blood pressure ≥ 140) OR (cholesterol ≥ 240)*”—a 2-condition Horn clause that a clinician can immediately validate against domain knowledge.

Finance and GDPR Compliance. The EU General Data Protection Regulation mandates a “right to explanation” for automated decisions affecting individuals (Article 22). TractTree’s polynomial-time AXp guarantee means that for every credit decision, every insurance assessment, and every fraud flag, a minimal sufficient explanation can be computed in bounded time—not as a post-hoc approximation, but as a formal certificate derived from the model’s structure.

Trustworthy AI. As machine learning systems are deployed in high-stakes domains, the ability to explain predictions is no longer optional. TractTree provides a model where accuracy, compactness, and explainability are not traded off against each other, but reinforced by the same architectural choices.

6.2 Limitations and Future Work

We note the following limitations in the spirit of research honesty:

- Results are validated under the described experimental protocol (10-run stratified split on 46 datasets). Broader generalization would be strengthened by additional protocols and larger-scale benchmarks.
- The current search over feature triples is cubic in the number of selected features ($O(\binom{K}{3} \cdot B^3)$). While the top- K pre-filtering ($K = 15$ by default) keeps this manageable, extending to 4-literal clauses would require additional pruning strategies.
- Affine (XOR) splits are most effective on binary or discretized features. Their utility on inherently continuous data with no parity structure is limited, though the BIC penalty correctly suppresses them in such cases.

Future work will explore integration of TractTree with ensemble methods (random forests, gradient boosting), where the tractability guarantees of individual trees can be preserved while further improving predictive performance.

References

- [1] C. E. Brodley and P. E. Utgoff, “Multivariate decision trees,” *Machine Learning*, vol. 11, pp. 5–31, 1992.
- [2] C. Carbonnel, M. C. Cooper, and B. Escoffier, “Tractable explaining of multi-valued decision trees,” *Proceedings of AAAI*, 2024.

- [3] P. Viola and M. J. Jones, “Rapid object detection using a boosted cascade of simple features,” *CVPR*, 2001.
- [4] G. Schwarz, “Estimating the dimension of a model,” *The Annals of Statistics*, vol. 6, no. 2, pp. 461–464, 1978.